UCS645: Parallel & Distributed Computing

LAB 1: Introduction to OpenMP

Lab Report

Name – Gagandeep

Roll no - 102303349

Q1: DAXPY Loop

Objective

DAXPY stands for Double precision A*X Plus Y. The operation performed is X[i] = a*X[i] + Y[i] on vectors of size 2^16. The goal is to compare speedup by increasing the number of threads.

Results

Threads	Execution Time (sec)	Speedup (T2/Tn)
2	0.002247	1.00 (baseline)
4	0.034476	0.07x (slower)
8	0.004417	0.51x

Observations

2 threads gave the minimum execution time of 0.002247 seconds, making it the optimal configuration.

4 threads showed a dramatic increase in time (0.034476s) due to excessive thread creation and synchronization overhead for this small workload.

8 threads improved slightly over 4 threads but remained slower than 2 threads due to limited CPU cores on the VirtualBox environment.

Conclusion

For the DAXPY loop with vector size 2^16, 2 threads provided maximum speedup. Beyond this, thread overhead exceeded computation gains due to limited CPU resources. This demonstrates that optimal thread count depends on problem size and available hardware cores.

Q2: Matrix Multiplication

Objective

Implement parallel matrix multiplication for 1000x1000 matrices using two approaches: 1D threading (parallelizing outer loop only) and 2D threading (parallelizing both loops using collapse(2)).

Results - 1D Threading

Threads	Execution Time (sec)	Speedup (T2/Tn)
2	0.357360	1.00 (baseline)
4	0.409395	0.87x (slower)
8	0.420742	0.85x (slower)

Results - 2D Threading

Threads	Execution Time (sec)	Speedup (T2/Tn)
2	0.170162	1.00 (baseline)
4	0.226810	0.75x

8	0.216421	0.79x

Comparison: 1D vs 2D

Threads	1D Time (sec)	2D Time (sec)	2D Improvement
2	0.357360	0.170162	2.10x faster
4	0.409395	0.226810	1.81x faster
8	0.420742	0.216421	1.94x faster

Observations

1D threading only parallelizes the outer (row) loop, leading to uneven workload distribution and poor performance beyond 2 threads.

2D threading with collapse(2) distributes both row and column iterations across threads, resulting in significantly better load balancing.

2D version was consistently ~2x faster than 1D version across all thread counts.

Both versions showed degrading performance beyond 2 threads, indicating VirtualBox CPU limitations.

Conclusion

2D parallelization using collapse(2) is clearly superior to 1D for matrix multiplication. It distributes work more evenly, reducing idle thread time. However, on a VirtualBox with limited cores, 2 threads remained optimal for both approaches.

Q3: Calculation of PI

Objective

Approximate the value of PI using numerical integration (Riemann sum). Multiple threads cooperate to compute the sum, using reduction to avoid race conditions.

Results

Threads	Calculated PI	Actual PI	Error	Time (sec)
2	3.141691	3.141593	0.000098	0.076509
4	3.141784	3.141593	0.000191	0.096210
8	3.142550	3.141593	0.000957	0.094454

Observations

2 threads gave the most accurate PI value (3.141691) and fastest execution time (0.076509s).

The reduction clause successfully prevented race conditions by giving each thread a private sum variable and combining them at the end.

More threads caused slightly less accurate PI values due to floating point summation ordering differences.

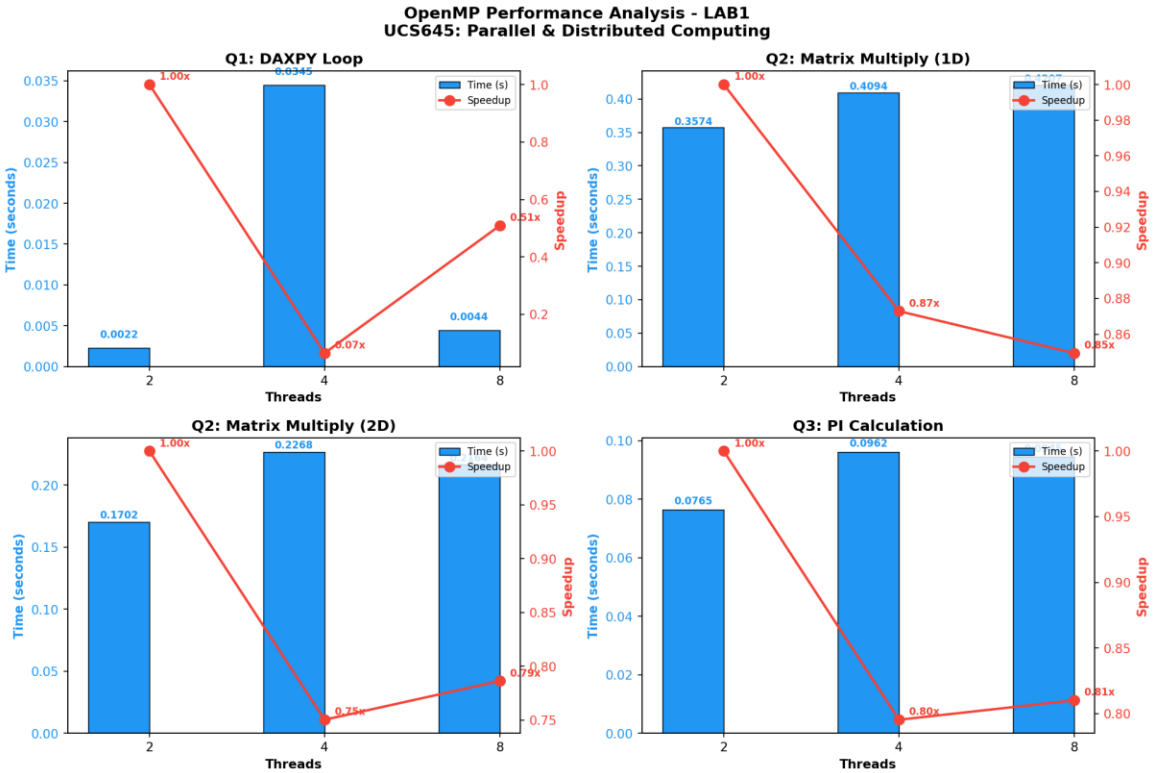
Performance degraded with more threads due to synchronization overhead on the VirtualBox environment.

Conclusion

The PI calculation using OpenMP reduction worked correctly and efficiently. The reduction clause is essential to avoid race conditions in parallel summation. Best accuracy and performance were achieved with 2 threads. This experiment demonstrated that for numerical computations, OpenMP reduction is the correct approach over critical sections.

Performance Graphs

The graphs below show execution time (bars) and relative speedup (line) for all three experiments across 2, 4, and 8 threads.



Overall Conclusion

This lab demonstrated the fundamentals of OpenMP parallel programming on a shared memory system using the Fork-Join model.

Key findings across all three experiments: 2 threads consistently provided the best performance on the VirtualBox environment due to limited physical CPU cores. Thread overhead exceeds computation gains for small-to-medium workloads when using more than 2 threads.

2D parallelization is significantly more efficient than 1D for matrix operations, achieving nearly 2x better performance by using collapse(2) to distribute both loop dimensions.

The reduction clause in OpenMP is the correct and efficient way to handle parallel summation, avoiding race conditions without the bottleneck of critical sections.

On a real multi-core machine with 8 or more physical cores, higher thread counts would demonstrate better speedup. The VirtualBox environment with shared CPU resources limits parallel performance, explaining why more threads did not always improve execution time.